C PROGRAMMING CO 2 ASSIGNMENT 3

NAME : BARANITHA A 1965260049

Q1 – Searching a Library Catalogue

```
#include <stdio.h>
int steps;
int binary_search(int a[], int n, int key)
{
    int low = 0, high = n - 1;
    steps = 0;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        steps++;

        if (a[mid] == key)
            return mid;
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main()
{
    int a[] = {1021, 1048, 1103, 1190, 1222,
               1305, 1377, 1402, 1460, 1533};
```

```
    int keys[] = {1021, 1305, 1533, 1400};
    int n = 10;

    for (int i = 0; i < 4; i++)
    {
        int pos = binary_search(a, n, keys[i]);
        printf("Book %d -> index %d, comparisons = %d\n",
              keys[i], pos, steps);
    }
    return 0;
}
```

OUTPUT :

```
Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4
```

Q2 – Sorting the Examination Hall Seating Register

```
#include <stdio.h>
int main()
{
   int a[6], b[6], c[6], temp, key;

   printf("Enter 6 numbers: ");
   for (int i = 0; i < 6; i++)
   {
      scanf("%d", &a[i]);
      b[i] = c[i] = a[i];
   }



   // Bubble Sort
   for (int i = 0; i < 5; i++)
      for (int j = 0; j < 5-i; j++)
         if (a[j] > a[j+1])
         {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
         }

   // Selection Sort
   for (int i = 0; i < 5; i++)
      for (int j = i+1; j < 6; j++)
         if (b[i] > b[j])
         {
            temp = b[i];
            b[i] = b[j];
```

```
            b[j] = temp;
        }

    // Insertion Sort
    for (int i = 1; i < 6; i++)
    {
        key = c[i];
        int j = i-1;

        while (j >= 0 && c[j] > key)
        {
            c[j+1] = c[j];
            j--;
        }
        c[j+1] = key;
    }
    printf("Bubble Sort: ");
    for (int i = 0; i < 6; i++) printf("%d ", a[i]);

    printf("\nSelection Sort: ");
    for (int i = 0; i < 6; i++) printf("%d ", b[i]);

    printf("\nInsertion Sort: ");
    for (int i = 0; i < 6; i++) printf("%d ", c[i]);

    return 0;
}
```

OUTPUT :

```
Enter 6 register numbers: 6
42 12 36 40 20 10
Bubble Sort: 6 12 20 36 40 42
Selection Sort: 6 12 20 36 40 42
Insertion Sort: 6 12 20 36 40 42
```

Q3 – Merging Two Merit Lists

```
#include <stdio.h>
int merge(int a[], int na, int b[], int nb, int out[])
{
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb)
    {
        if (a[i] >= b[j])
            out[k++] = a[i++];
        else
            out[k++] = b[j++];
    }
    while (i < na)
        out[k++] = a[i++];
    while (j < nb)
        out[k++] = b[j++];
    return k;
}
int main()
{
    int a[] = {98, 91, 85, 72};
    int b[] = {95, 91, 80};
    int out[7];
    int n = merge(a, 4, b, 3, out);
    printf("Merged Merit List: ");
    for (int i = 0; i < n; i++)
        printf("%d ", out[i]);
    return 0;
}
```

OUTPUT :

```
Merged Merit List: 98 95 91 91 85 80 72
```

Q4 – Two-Dimensional Arrays: District Rainfall Records

```
#include <stdio.h>
int main()
{
   int rain[3][4];
   int i, j, total, max = 0, maxRow = 0, maxCol = 0;
   float sum = 0;
   printf("Enter rainfall values (3x4):\n");
   for (i = 0; i < 3; i++)
   {
      for (j = 0; j < 4; j++)
      {
         scanf("%d", &rain[i][j]);
         sum += rain[i][j];
         if (rain[i][j] > max)
         {
            max = rain[i][j];
            maxRow = i;
            maxCol = j;
         }
      }
   }

   for (i = 0; i < 3; i++)
   {
      total = 0;
      for (j = 0; j < 4; j++)
         total += rain[i][j];

      printf("District %d Total = %d\n", i + 1, total);
   }
printf("Monthly Averages:\n");
```

```
    for (j = 0; j < 4; j++)
    {
        float avg = 0;

        for (i = 0; i < 3; i++)
            avg += rain[i][j];

        printf("%.2f ", avg / 3);
    }

    printf("\nState Average = %.2f", sum / 12);
    printf("\nMaximum Rainfall = %d", max);
    printf("\nDistrict = %d, Month = %d", maxRow + 1, maxCol + 1);

    return 0;
}
```

OUTPUT:

```
Enter rainfall values (3x4):
10 20 30
40 50 60
70 80 90
100 110 120
District 1 Total = 100
District 2 Total = 260
District 3 Total = 420
Monthly Averages:
50.00 60.00 70.00 80.00
State Average = 65.00
Maximum Rainfall = 120
District = 3, Month = 4
```

Q5 – Processing Text Without the Library

```
#include <stdio.h>
#include <ctype.h>
int main()
{
    char s[] = "Madam";
    int i, n = 0, flag = 1, words = 0;
    for (i = 0; s[i] != '\0'; i++)
        n++;
    for (i = 0; i < n; i++)
    {
        if (i == 0 || s[i-1] == ' ')
            words++;
    }
    for (i = 0; i < n/2; i++)
    {
        if (tolower(s[i]) != tolower(s[n-i-1]))
            flag = 0;
    }
    printf("Words: %d\n", words);
    printf("Palindrome: %d", flag);
    return 0;
}
```

OUTPUT :

```
Words: 7
1 0
```

Q6 – A Directory of Names: Sorting and Searching Strings

```
#include <stdio.h>
#include <string.h>
int main()
{
    char names[10][30], temp[30], key[30];
    int n, i, j, low, high, mid, found = 0;
    printf("Enter number of names: ");
    scanf("%d", &n);
    getchar();
    printf("Enter names:\n");
    for (i = 0; i < n; i++)
    {
        fgets(names[i], 30, stdin);
        names[i][strcspn(names[i], "\n")] = '\0';
    }
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (strcmp(names[j], names[j + 1]) > 0)
            {
                strcpy(temp, names[j]);
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }
    printf("Sorted Names:\n");
    for (i = 0; i < n; i++)
        printf("%s\n", names[i]);
```

```
    printf("Duplicate Names:\n");
    for (i = 0; i < n - 1; i++)
    {
        if (strcmp(names[i], names[i + 1]) == 0)
            printf("%s\n", names[i]);
    }
    printf("Enter name to search: ");
    fgets(key, 30, stdin);
    key[strcspn(key, "\n")] = '\0';

    low = 0;
    high = n - 1;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (strcmp(names[mid], key) == 0)
        {
            found = 1;
            break;
        }
        else if (strcmp(names[mid], key) < 0)
            low = mid + 1;
        else
            high = mid - 1;
    }
    if (found)
        printf("Name Found");
    else
        printf("Name Not Found");
    return 0;
}
```

OUTPUTS :

```
Enter number of names: 5
Enter names:
Anu
Hema
Jeni
Vib
Yaz
Sorted Names:
Anu
Hema
Jeni
Vib
Yaz
Duplicate Names:
Enter name to search: Vib
Name Found
```

```
Enter number of names: 3
Enter names:
ANU
HARINI
YAZH
Sorted Names:
ANU
HARINI
YAZH
Duplicate Names:
Enter name to search: BALA
Name Not Found
```

Q7 – Design an Election Counting Program

```
#include <stdio.h>

int main()
{
    int votes[100], count[12] = {0};
    int n, c, valid = 0, invalid = 0;
    int i, j, temp, id[12], max = 0, tie = 0;

    printf("Enter candidates and votes: ");
```

```
scanf("%d%d", &c, &n);

for (i = 0; i < n; i++)
{
   scanf("%d", &votes[i]);

   if (votes[i] >= 1 && votes[i] <= c)
   {
      count[votes[i] - 1]++;
      valid++;
   }
   else
      invalid++;
}

for (i = 0; i < c; i++)
   id[i] = i + 1;

for (i = 0; i < c - 1; i++)
{
   for (j = 0; j < c - i - 1; j++)
   {
      if (count[j] < count[j + 1])
      {
         temp = count[j];
         count[j] = count[j + 1];
         count[j + 1] = temp;

         temp = id[j];
         id[j] = id[j + 1];
         id[j + 1] = temp;
      }
```

```
        }
    }

    printf("Valid Votes = %d\n", valid);
    printf("Invalid Votes = %d\n", invalid);
    printf("Candidate  Votes\n");

    for (i = 0; i < c; i++)
    {
        printf("%d          %d\n", id[i], count[i]);

        if (count[i] > max)
            max = count[i];
    }

    for (i = 0; i < c; i++)
        if (count[i] == max)
            tie++;
    if (tie > 1)
        printf("Election Result: Tie");
    else
        printf("Winner: Candidate %d", id[0]);

    return 0;
}
```

OUTPUT :

```
Enter candidates and votes: 3 8
1 2 1 3 2 1 4 0
Valid Votes = 6
Invalid Votes = 2
Candidate  Votes
1          3
2          2
3          1
Winner: Candidate 1
```